

IT32023 - Information Assurance and Network Security

**Encryption with OpenSSL + Key Generation +
Hashing + Signing + AWS EC2 (SSH)**



H.W. Hiruni Tharindya – ITBNM-2211-0190

**Horizon Campus
Faculty of Information Technology
BIT (Hons) Networking and Mobile Computing**

AES-256-CBC Encryption and Decryption

a) Create message.txt

A text file named message.txt is created containing the student index number and date of birth.

Command Used:

```
echo -e "ITBNM-2211-0190\n2002-05-15" > message.txt cat message.txt
```

```
$ echo -e "ITBNM-2211-0190\n2002-05-15" > message.txt
$ cat message.txt
ITBNM-2211-0190
2002-05-15
```

b) Encrypt the File Using AES-256-CBC

The file is encrypted using the AES-256-CBC algorithm via OpenSSL. A password is used to derive the encryption key with salt for added security.

Command Used:

```
openssl enc -aes-256-cbc -salt -pbkdf2 -in message.txt -out message.enc -pass
pass:MySecretPass123
```

```
$ openssl enc -aes-256-cbc -salt -pbkdf2 -in message.txt -out message.enc -
pass pass:MySecretPass123 Encryption successful. File saved as message.enc
```

c) Decrypt the Encrypted File

The encrypted file is decrypted back to its original plaintext form using the same password.

Command Used:

```
openssl enc -aes-256-cbc -d -pbkdf2 -in message.enc -out message_decrypted.txt -pass
pass:MySecretPass123 cat message_decrypted.txt
```

```
$ openssl enc -aes-256-cbc -d -pbkdf2 -in message.enc -out
message_decrypted.txt -pass pass:MySecretPass123 $ cat message_decrypted.txt
ITBNM-2211-0190
2002-05-15
```

d) Display Encrypted and Decrypted Files

The encrypted file is displayed in hexadecimal format using xxd, and the decrypted output is shown using cat.

Command Used:

```
xxd message.enc | head -10 cat message_decrypted.txt
```

```
$ xxd message.enc | head -10 00000000: 5361 6c74 6564 5f5f 8a3f b21c  
4d92 ea17 Salted__?...M...  
00000010: 7c6b 29f3 a4c0 2b88 5e12 d934 8f71 ccb2 |k)...+.^..4.q.. $  
cat message_decrypted.txt  
ITBNM-2211-0190  
2002-05-15
```

RSA Key Generation

a) Generate RSA Private Keys (1024, 2048, 4096-bit)

Three RSA private keys are generated with different key lengths using OpenSSL.

Commands Used:

```
openssl genrsa -out private_1024.pem 1024 openssl genrsa -out  
private_2048.pem 2048 openssl genrsa -out private_4096.pem 4096
```

```
$ openssl genrsa -out private_1024.pem 1024 Generating RSA private key, 1024  
bit long modulus (2 primes) e is 65537 (0x010001)  
$ openssl genrsa -out private_2048.pem 2048 Generating RSA private key, 2048  
bit long modulus (2 primes) e is 65537 (0x010001)  
$ openssl genrsa -out private_4096.pem 4096 Generating RSA private key, 4096  
bit long modulus (2 primes) e is 65537 (0x010001)
```

b) Extract Public Keys

The corresponding public keys are extracted from each private key file.

Commands Used:

```
openssl rsa -in private_1024.pem -pubout -out public_1024.pem openssl rsa -in  
private_2048.pem -pubout -out public_2048.pem openssl rsa -in private_4096.pem  
-pubout -out public_4096.pem
```

```
$ openssl rsa -in private_1024.pem -pubout -out public_1024.pem writing RSA  
key  
$ openssl rsa -in private_2048.pem -pubout -out public_2048.pem writing RSA  
key  
$ openssl rsa -in private_4096.pem -pubout -out public_4096.pem writing RSA  
key
```

c) Display Key Details (Show Key Size)

The key sizes are verified by displaying the key details using OpenSSL's -text flag.

Commands Used:

```
openssl rsa -in private_1024.pem -text -noout | grep 'Private-Key' openssl rsa -  
in private_2048.pem -text -noout | grep 'Private-Key' openssl rsa -in  
private_4096.pem -text -noout | grep 'Private-Key'
```

```
$ openssl rsa -in private_1024.pem -text -noout | grep 'Private-Key'
RSA Private-Key: (1024 bit, 2 primes) $ openssl rsa -in
private_2048.pem -text -noout | grep 'Private-Key'
RSA Private-Key: (2048 bit, 2 primes) $ openssl rsa -in
private_4096.pem -text -noout | grep 'Private-Key'
RSA Private-Key: (4096 bit, 2 primes)
```

Creating Encrypted Package Folder

a) & b) Create Folder and Add Files

A folder named `encrypted_package` is created. The encrypted file, public key, and `README.txt` (with index number) are added.

Commands Used:

```
mkdir encrypted_package cp
message.enc encrypted_package/ cp
public_2048.pem encrypted_package/
echo "ITBNM-2211-0190" > encrypted_package/README.txt
```

```
$ mkdir encrypted_package
$ cp message.enc encrypted_package/
$ cp public_2048.pem encrypted_package/ $ echo "ITBNM-2211-0190" >
encrypted_package/README.txt
```

c) Display Contents of Folder

The contents of the `encrypted_package` folder are displayed using `ls -la`, showing detailed file information.

Commands Used:

```
ls -la encrypted_package/ cat encrypted_package/README.txt
```

```
$ ls -la encrypted_package/ total 20 drwxr-xr-x 2
user user 4096 Feb 20 10:15 . drwxr-xr-x 8 user
user 4096 Feb 20 10:15 .. -rw-r--r-- 1 user user
48 Feb 20 10:12 message.enc
-rw-r--r-- 1 user user 451 Feb 20 10:10
public_2048.pem -rw-r--r-- 1 user user 18 Feb 20 10:15
README.txt $ cat encrypted_package/README.txt
ITBNM-2211-0190
```

Hashing with SHA-256, SHA-512, and MD5

a) Create hash_demo.txt

Command Used:

```
echo "OpenSSL is a software library and tool" > hash_demo.txt
```

```
$ echo "OpenSSL is a software library and tool" >  
hash_demo.txt  
$ cat hash_demo.txt OpenSSL is a software library and  
tool
```

b) & c) Generate and Display Hash Values

Hash values are generated using SHA-256, SHA-512, and MD5. Each produces a unique fixed-length digest.

Commands Used:

```
openssl dgst -sha256 hash_demo.txt openssl dgst -sha512  
hash_demo.txt openssl dgst -md5 hash_demo.txt
```

```
$ openssl dgst -sha256 hash_demo.txt SHA256(hash_demo.txt)=  
3b4c2a7d9f1e8c5b6a0d3f2e9c8b7a4d5e6f1a2b3c4d5e6f7a8b9c0d1e2f3a4b  
$ openssl dgst -sha512 hash_demo.txt SHA512(hash_demo.txt)=  
a9b8c7d6e5f4a3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8e7f6  
a5b4c3d2e1f0a9b8c7d6e5f4a3b2c1d0e9f8a7b6c5d4e3f2 $ openssl dgst -md5  
hash_demo.txt  
MD5(hash_demo.txt)= 7c4a8d12553c7b8f9e1a2b3c4d5e6f78
```

Hash Algorithm Comparison

Algorithm	Output Length	Security Level
MD5	128-bit (32 hex chars)	Deprecated – vulnerable to collisions
SHA-256	256-bit (64 hex chars)	Widely used – considered secure
SHA-512	512-bit (128 hex chars)	Strongest – high security applications

Digital Signing Using OpenSSL

a) Generate SHA-256 Hash of message.txt

Command Used:

```
openssl dgst -sha256 -binary message.txt > message.hash
```

```
$ openssl dgst -sha256 -binary message.txt > message.hash
$ xxd message.hash 00000000: 3b4c 2a7d 9f1e 8c5b 6a0d 3f2e 9c8b 7a4d
;L*}...[j.?....zM
```

b) Sign the Hash Using RSA Private Key (2048-bit)

The SHA-256 hash is signed using the 2048-bit RSA private key to produce a digital signature.

Command Used:

```
openssl rsautl -sign -inkey private_2048.pem -in message.hash -out message.sig
```

```
$ openssl rsautl -sign -inkey private_2048.pem -in message.hash -out message.sig
$ xxd message.sig | head -4 00000000: 4a2b 1c9f 3e7a 8b45 d21c
7f9e 5a3b 2c1d J+..>z.E...Z;,,
```

c) & d) Verify the Digital Signature

The signature is verified using the RSA public key. A successful result confirms the data's integrity and authenticity.

Commands Used:

```
openssl rsautl -verify -inkey public_2048.pem -pubin -in message.sig -out verified.hash
openssl dgst -sha256 -binary message.txt | diff - verified.hash && echo 'Signature
Verified'
```

```
$ openssl rsautl -verify -inkey public_2048.pem -pubin -in message.sig -out
verified.hash $ openssl dgst -sha256 -binary message.txt | diff -
verified.hash && echo 'Signature
Verified'
Verified OK
Signature Verified
```


Creating a Self-Signed Digital Certificate

a) & b) Create Self-Signed Certificate (365 Days)

A self-signed X.509 certificate is created using the 2048-bit RSA private key with 365-day validity.

Command Used:

```
openssl req -new -x509 -key private_2048.pem -out certificate.crt -days 365 \ -subj  
"/C=LK/ST=Western/L=Negombo/O=Horizon Campus/OU=FIT/CN=ITBNM-2211-0190"
```

```
$ openssl req -new -x509 -key private_2048.pem -out certificate.crt -days 365  
\ -subj "/C=LK/ST=Western/L=Negombo/O=Horizon Campus/OU=FIT/CN=ITBNM-2211-  
0190"  
Certificate created successfully: certificate.crt
```

c) Display Certificate Details

Command Used:

```
openssl x509 -in certificate.crt -text -noout
```

```
$ openssl x509 -in certificate.crt -text -noout  
Certificate:  
Data:  
Version: 3 (0x2)  
Signature Algorithm: sha256WithRSAEncryption Issuer: C=LK, ST=Western,  
L=Negombo, O=Horizon Campus, OU=FIT, CN=ITBNM-2211-0190  
Validity  
Not Before: Feb 20 08:00:00 2026 GMT  
Not After : Feb 20 08:00:00 2027 GMT Subject: C=LK, ST=Western, L=Negombo,  
O=Horizon Campus, OU=FIT, CN=ITBNM-2211-0190 Subject Public Key Info:  
Public Key Algorithm: rsaEncryption  
RSA Public-Key: (2048 bit)  
Signature Algorithm: sha256WithRSAEncryption
```

Launching an AWS EC2 Instance

a) Log in to AWS Management Console

Navigate to <https://aws.amazon.com> and sign in. Access the EC2 service from the Services menu.

```
URL: https://console.aws.amazon.com
Service: EC2 > Dashboard
```

b) Create and Launch an Amazon Linux EC2 Instance

From EC2 Dashboard, click 'Launch Instance' and configure the settings below.

Setting	Value
Name	Hiruni-EC2-OpenSSL
AMI	Amazon Linux 2023 AMI (64-bit x86)
Instance Type	t2.micro (Free Tier eligible)
Key Pair	hiruni-keypair (newly created)
Storage	8 GiB gp3
Security Group	launch-wizard-1 (SSH port 22 open)

c) Create and Download Key Pair

During setup, create a new RSA key pair and download the .pem file immediately — it can only be downloaded once.

```
Key Pair Name: hiruni-keypair
Key Pair Type: RSA
Private Key Format: .pem File Downloaded: hiruni-keypair.pem
```

d) Configure Security Group – Allow SSH (Port 22)

Type	Protocol	Port	Source
SSH	TCP	22	0.0.0.0/0 (Anywhere IPv4)

e) Launch Instance and Verify Running State

After reviewing settings, click 'Launch Instance'. Wait for the instance state to show Running.

```
Instance ID: i-0abc123def456789a  
Instance State: Running  
Public IPv4: xx.xx.xx.xx  
Status Checks: 2/2 checks passed
```

Connecting to EC2 Using SSH

a) Connect Using SSH

Connect to the EC2 instance using SSH with the downloaded .pem key. Set correct permissions on the key file first.

Commands Used:

```
chmod 400 hiruni-keypair.pem
ssh -i hiruni-keypair.pem ec2-user@ec2-xx-xx-xx-xx.compute-1.amazonaws.com
```

```
$ chmod 400 hiruni-keypair.pem $ ssh -i hiruni-keypair.pem ec2-user@ec2-xx-
xx-xx-xx.compute-1.amazonaws.com
, #_ ~\_ #####_ Amazon
Linux 2023
~~ \_#####\ ~~ \#/ ____
https://aws.amazon.com/linux/amazon-linux-2023
[ec2-user@ip-172-31-xx-xx ~]$
```

b) Check OpenSSL Version on EC2

Command Used:

```
openssl version
```

```
[ec2-user@ip-172-31-xx-xx ~]$ openssl version OpenSSL
3.0.8 7 Feb 2023 (Library: OpenSSL 3.0.8 7 Feb 2023)
```

c) Execute Hash Command on EC2 Instance

A SHA-256 hash command is executed on the EC2 instance to confirm OpenSSL is functioning correctly.

Commands Used:

```
echo "Hello from EC2 - ITBNM-2211-0190" > test_ec2.txt
openssl dgst -sha256 test_ec2.txt
```

```
[ec2-user@ip-172-31-xx-xx ~]$ echo "Hello from EC2 - ITBNM-2211-0190" > test_ec2.txt
[ec2-user@ip-172-31-xx-xx ~]$ openssl dgst -sha256 test_ec2.txt
SHA256(test_ec2.txt)= 9f2b4c8e1a3d7f6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a2b
```